minicollide: Numerical Methods for Real-Time Rigid-Body Dynamics in a Compact, Deterministic Physics Engine

Tomas Jimenez
rigid-body-lab
Code: https://github.com/tomjnet/rigid-body-lab

***Abstract*—We present *minicollide*, a from-scratch 3D rigid-body physics engine (∼3k lines of dependency-free C++20) built as a laboratory for the numerical methods behind real-time simulation. The engine implements symplectic integration of the Newton–Euler equations, a sweep-and-prune broadphase, exact and iterative narrowphase collision detection (SAT with polygon clipping, GJK+EPA), and a sequential-impulse constraint solver (projected Gauss–Seidel) with warm starting and Baumgarte stabilization. Constraint islands are extracted by union-find and solved in parallel while preserving bit-exact determinism across thread counts. We describe the mathematics of each stage and validate the engine with four instrumented demos: box stacking, jointed chains, an interactive destruction sandbox, and an emergent molecular-bonding simulation driven entirely by the engine's contact manifolds.**

I. Introduction

Real-time physics engines solve a constrained dynamics problem at fixed frequency (here 60 Hz) under a hard budget of a few milliseconds per step. This forces numerical choices that differ sharply from offline multibody simulation: fixed timesteps, iterative solvers truncated far from convergence, and aggressive exploitation of temporal coherence. *minicollide* was written to study these choices by implementing each one from the original literature. The pipeline per step is: integrate velocities → broadphase → narrowphase → manifold persistence → island decomposition → per-island constraint solve → integrate positions → sleeping.

II. Equations of Motion and Integration

Each body carries linear and angular state $(\mathbf{x}, \mathbf{q}, \mathbf{v}, \boldsymbol{\omega})$ with orientation stored as a unit quaternion. The Newton–Euler equations are integrated with semi-implicit (symplectic) Euler: velocities first, then positions with the *new* velocity,

$$\mathbf{v}^{+} = \mathbf{v} + h\, m^{-1}\mathbf{f}, \qquad \boldsymbol{\omega}^{+} = \boldsymbol{\omega} + h\, \mathbf{I}^{-1}\boldsymbol{\tau}, \tag{1}$$

$$\mathbf{x}^{+} = \mathbf{x} + h\, \mathbf{v}^{+}, \qquad \mathbf{q}^{+} = \mathrm{normalize}\big(\mathbf{q} + \tfrac{h}{2}\,\tilde{\boldsymbol{\omega}}^{+}\mathbf{q}\big), \tag{2}$$

where $\tilde{\boldsymbol{\omega}}$ is the pure quaternion $(0, \boldsymbol{\omega})$. Symplectic Euler costs the same as explicit Euler but exhibits bounded energy behavior on oscillatory systems, making it the de facto game-physics integrator. The world-space inverse inertia tensor is rebuilt every step from the body rotation R, $\mathbf{I}_w^{-1} = R\,\mathbf{I}_\ell^{-1}\,R^{\mathsf{T}}$, with $\mathbf{I}_\ell$ the diagonal inertia of the collision shape (sphere, box, or capsule; the capsule's tensor composes a cylinder with two hemispherical caps, mass split by volume).

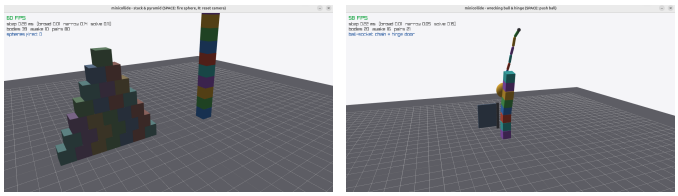


Fig. 1. Left: `demo_stack` — a 10-box tower and 5-layer pyramid resting without jitter at 60 FPS (0.28 ms/step); the pyramid sleeps as one island (desaturated). Right: `demo_chain` — a wrecking ball on a 7-link ball-socket chain beside a hinged door; joints and contacts interleave in the same island solve.

III. Collision Detection

Broadphase. Axis-aligned bounding boxes, fattened by a margin, are sorted along x and swept ($O(n \log n)$ sort, near-linear sweep), emitting candidate pairs whose intervals overlap on all three axes.

Box–box narrowphase. The separating axis theorem is evaluated over the 15 candidate axes of two oriented boxes (6 face normals, 9 edge–edge cross products), with a small bias preferring face axes for manifold robustness. Given the minimizing face axis, the incident face is clipped against the side planes of the reference face (Sutherland–Hodgman), yielding up to four contact points. Multi-point manifolds are what make stacking stable (Fig. 1): a single-point manifold cannot resist the tipping torque of a resting box.

GJK and EPA. For capsule–box, distance queries run Gilbert–Johnson–Keerthi [3] on the Minkowski difference of the capsule's inner segment and the box, using barycentric closest-point-on-simplex to extract witness points; on overlap, the Expanding Polytope Algorithm grows the terminal simplex into a polytope whose closest face gives penetration normal and depth. The capsule radius is added back afterwards — van den Bergen's shrunk-shape margin trick [4].

Persistent manifolds. Contact points are matched frame-to-frame by local-anchor proximity so accumulated impulses survive, enabling the warm starting described next.

IV. Constraint Solving

Contacts and joints are velocity constraints $J\mathbf{v} + b = 0$ solved by *sequential impulses* [1], algebraically equivalent to projected Gauss–Seidel on the mixed linear complementarity

problem. For each contact point with Jacobian J the effective mass is

$$m_{\text{eff}} = \left(J\,M^{-1}J^{\mathsf{T}}\right)^{-1}, \tag{3}$$

and each iteration applies $\Delta\lambda = -m_{\text{eff}}\,(J\mathbf{v} + b)$. The key convergence device is *accumulated-impulse clamping*: the increment may be negative as long as the running total satisfies $\lambda_n \geq 0$ (normal) and $|\lambda_t| \leq \mu\lambda_n$ (friction, clamped per tangent as a box approximation of the Coulomb cone). Warm starting re-applies last frame's converged impulses before iterating, exploiting temporal coherence; the engine's 5-box stack test fails without it. Position drift is controlled by a Baumgarte [2] bias $b = -\frac{\beta}{h}\max(d - d_{\text{slop}}, 0)$ folded into the velocity constraint, and restitution is applied only above a speed threshold to keep resting contacts quiet.

Joints. A ball-socket joint constrains the anchor points of two bodies to coincide; its three linear rows are solved as a block with the 3×3 effective-mass matrix

$$K = \left(\tfrac{1}{m_A} + \tfrac{1}{m_B}\right) E + \tilde{r}_A \mathbf{I}_A^{-1} \tilde{r}_A^{\mathsf{T}} + \tilde{r}_B \mathbf{I}_B^{-1} \tilde{r}_B^{\mathsf{T}}, \tag{4}$$

where $\tilde{r}$ is the cross-product (skew) matrix of the anchor arm. A hinge adds two angular rows over a basis perpendicular to the axis, solved with a 2×2 block. Joints interleave with contacts inside each island iteration (Fig. 1, right). Ten velocity iterations per island suffice for the scenes shown; the classic PGS failure mode — slow convergence at large mass ratios — is why the molecular demo scales element masses instead of using true atomic weights.

V. Parallelism and Determinism

Islands are connected components of the constraint graph over dynamic bodies, extracted by union-find (static bodies do not merge components). Islands are independent subproblems, so they are solved in parallel on a thread pool; the narrowphase parallelizes over candidate pairs. Determinism is preserved under threading by indexing all work by input order — never appending in completion order — and fixing each island's internal solve order: two runs at any thread count produce bit-identical state, verified by hashing in CI. Sleeping is island-granular: a body sleeps only when its whole island falls below velocity thresholds for 0.5 s, and any woken member wakes the island. A companion AVX2 benchmark vectorizes the normal-impulse row update 8 contacts wide over SoA data ($\sim$6$\times$ scalar), and a per-phase profiler exposes the Amdahl bottleneck: on a 1300-body scene the solve speeds up 1.8$\times$ on 4 cores but the total only 1.3$\times$, because the broadphase sort remains serial.

VI. Case Studies

The demos exercise the engine as validation scenes (Figs. 1–2). *Stacking* stresses manifold quality, warm starting, and Baumgarte tuning; the tower rests without jitter and sleeps. *Chain* couples high-mass-ratio joints (a 25 kg ball on 0.8 kg links) with contacts. *Fort* drives the engine as a game with no engine changes: a capsule character with zero inverse inertia, grounding read from contact manifolds, and pooled projectiles. *Molecules* shows the engine's outputs feeding back into its inputs: each frame, contact manifolds between atoms with free valence spawn ball-socket "covalent bonds" at the contact point, and molecules (H_2O, CO_2, CH_4) emerge without any dedicated chemistry code. A networked mode (authoritative 60 Hz server, 20 Hz UDP snapshots, client-side snapshot interpolation [5]) reuses the same deterministic core.

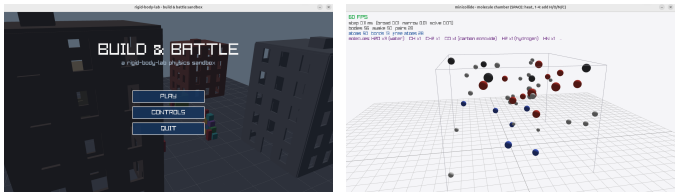


Fig. 2. Left: `demo_fort` — a build-and-battle sandbox with destructible structures; panels at zero hit points flip from static to dynamic and collapse under the solver. Right: `demo_molecules` — atoms with chemical valence bond on contact via runtime-created ball-socket joints; the HUD census (union-find over the bond graph) reports emergent H_2O, CO, and H_2 molecules.

VII. Conclusion

A complete, stable, parallel rigid-body engine fits in a few thousand lines once the right numerical methods are composed: symplectic integration, multi-point persistent manifolds, and clamped accumulated impulses with warm starting do the heavy lifting. Planned deep-dives include split-impulse position correction, speculative contacts for continuous collision, graph-colored SIMD solving in the engine loop, and Featherstone articulations. Source code, tests, and benchmarks: https://github.com/tomjnet/rigid-body-lab.

References

[1] E. Catto, "Iterative dynamics with temporal coherence," *Game Developers Conference*, 2005.

[2] J. Baumgarte, "Stabilization of constraints and integrals of motion in dynamical systems," *Comput. Methods Appl. Mech. Eng.*, vol. 1, pp. 1–16, 1972.

[3] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE J. Robot. Autom.*, vol. 4, no. 2, 1988.

[4] G. van den Bergen, *Collision Detection in Interactive 3D Environments*. Morgan Kaufmann, 2003.

[5] G. Fiedler, "Snapshot interpolation," *Gaffer on Games*, 2014.